

Homework5

- a. how to create a thread (4 ways, write code)

Extending the Thread Class

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Thread running: " + Thread.currentThread().getName());  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start(); // starts a new thread  
    }  
}
```

Implementing the Runnable Interface

```
class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("Runnable running: " + Thread.currentThread().getName());  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start();  
    }  
}
```

Using a Lambda Expression (Anonymous Runnable)

```
public class Main {  
    public static void main(String[] args) {  
        Thread t = new Thread() -> {  
            System.out.println("Lambda thread running: " +  
                Thread.currentThread().getName());  
        };  
        t.start();  
    }  
}
```

Using ExecutorService (Thread Pool)

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 5; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " on: " +
                    Thread.currentThread().getName());
            });
        }

        executor.shutdown(); // waits for all tasks to finish
    }
}

```

- b. thread lifecycle, how does thread transfer from one state to another

NEW → **RUNNABLE** — calling `t.start()` allocates an OS thread and moves it into the runnable pool.

BLOCKED: when a thread tries to enter a synchronized block or method but another thread already holds that monitor lock. It waits until the lock is free. → **RUNNABLE** — the moment the holding thread exits the synchronized block and releases the lock.

WAITING: calling `object.wait()`, `thread.join()`, or `LockSupport.park()` with no timeout. The thread suspends indefinitely and releases any held lock. It will wait forever unless explicitly woken. → **RUNNABLE**: another thread calls `object.notify()` / `notifyAll()`, or `LockSupport.unpark()`. The thread re-acquires the lock before continuing — always re-check the condition in a while loop, not if.

TIMED_WAITING: calling `Thread.sleep(ms)`, `object.wait(ms)`, `thread.join(ms)`, or `LockSupport.parkNanos()`. Same as **WAITING** but has a built-in expiry timer. → **RUNNABLE**: either the timeout expires naturally, or another thread calls `notify()` / `interrupt()` before the timeout. `sleep()` can only be woken by `interrupt()`.

RUNNABLE → **TERMINATED**: `run()` returns normally, or an unhandled exception propagates out of `run()`. A terminated thread cannot be restarted — you must create a new Thread object.

- c. how does thread pool work

Thread pool data type: `ExecutorService`

Thread pool is a container to contain multiple threads before loading threads after that thread stay inside the pool when a task need execute grab from pool and execute the task. After finish executed put back to the thread pool.

execute () submit the task to thread pool → thread go to `corePool` → go to the tail of `corePool` → got to the waiting Queue → check the maximum pool → create a thread → execute the task in waiting Queue

Thread pool has two parts:

Threadpool executor:

- Core pool: fix the number of all threads configured in core pool must be there until java program exit. Check from thread 0, if it is occupied check thread 1...
- workQueue: waiting bench used to assign jobs to thread
- Maximum pool: thread waiting for jobs until `keepAlivetime` be recycled

- d. what is the potential problem for the `newCachedThreadPool` and `newFixedThreadPool` and why
`newCachedThreadPool()` can create an unlimited number of threads, which may exhaust CPU and memory resources under heavy load.
`newFixedThreadPool()` limits the number of threads but uses an unbounded queue, so tasks can accumulate indefinitely and consume large amounts of memory. Both can potentially lead to `OutOfMemoryError` if not used carefully.
- e. what is Future
`Future<T>` represents the result of an asynchronous computation that may not be complete yet. You submit a task to an `ExecutorService` and get back a Future — you can later call `.get()` to retrieve the result, which **blocks** the calling thread until done.
- f. what is `CompletableFuture`
`CompletableFuture` is used for asynchronous and non-blocking programming. It supports chaining, combining, and callback-based handling — you don't need to block to get the result. It can also be **manually completed** with any value.
- ```
CompletableFuture.supplyAsync(() -> fetchUserFromDB(1)) // runs async
 .thenApply(user -> enrichWithProfile(user)) // transform result
 .thenCompose(user -> fetchOrdersAsync(user.getId())) // chain another async call
 .thenCombine(fetchInventoryAsync(), (orders, inv) -> merge(orders, inv)) // combine
two futures
 .thenAccept(result -> System.out.println("Done: " + result)) // consume result
 .exceptionally(ex -> { System.out.println("Error: " + ex); return null; }); // handle error
```
- g. Future vs `CompletableFuture`  
Future — blocking, limited. It has no support for chaining, callbacks, or combining multiple futures.

CompletableFuture — non-blocking, chainable. It supports chaining, combining, and callback-based handling — you don't need to block to get the result. It can also be **manually completed** with any value.

h. Lock vs synchronized

These are used for avoiding race condition and ensure thread safety by allowing only one thread to access a critical section at a time.

**Lock** is a class that

**Synchronized** put method scope but not parallel processing

is used for multithreading use synchronized block or synchronized method.

Only minimum critical section control -> synchronized block

i. what is wait(), notify(), notifyAll(), join()

- **wait()** makes the current thread release the object lock and wait until another thread calls notify() or notifyAll().
- **notify()** wakes up one waiting thread.
- **notifyAll()** wakes up all waiting threads.
- **join()** makes one thread wait for another thread to finish. For example, if main calls t.join(), the main thread waits until thread t completes.

j. commonly used CompletableFuture api and write demo code for each of them

i. **runAsync()** runs a task asynchronously but does not return a value.

**supplyAsync()** runs a task asynchronously and returns a result.

```
import java.util.concurrent.CompletableFuture;
```

```
public class RunVsSupply {
 public static void main(String[] args) throws Exception {
```

```
 // runAsync — no return value (Runnable)
```

```
 CompletableFuture<Void> run = CompletableFuture.runAsync(() -> {
```

```
 System.out.println("runAsync: sending email on thread "
```

```
 + Thread.currentThread().getName());
```

```
 });
```

```
 run.get(); // wait for completion
```

```
 // supplyAsync — returns a value (Supplier)
```

```
 CompletableFuture<String> supply = CompletableFuture.supplyAsync(() ->
```

```
{
```

```
 System.out.println("supplyAsync: fetching user on thread "
```

```
 + Thread.currentThread().getName());
```

```
 return "Alice";
```

```
});
```

```
String user = supply.get();
```

```

 System.out.println("supplyAsync result: " + user);
 }
}

```

- ii. **thenApply()** transforms the result of a previous task.  
**thenApplyAsync()** also transforms the result, but it may run in another thread.

```

public class ThenApplyDemo {
 public static void main(String[] args) throws Exception {

 // thenApply — transforms result in the SAME thread (or calling thread)
 CompletableFuture<String> apply = CompletableFuture
 .supplyAsync(() -> "alice") // worker thread
 .thenApply(name -> {
 System.out.println("thenApply on: " +
Thread.currentThread().getName());
 return name.toUpperCase(); // "ALICE"
 })
 .thenApply(name -> "Hello, " + name); // "Hello, ALICE"

 System.out.println(apply.get());

 // thenApplyAsync — transforms result, but ALWAYS in a new thread
 CompletableFuture<String> applyAsync = CompletableFuture
 .supplyAsync(() -> "bob")
 .thenApplyAsync(name -> {
 System.out.println("thenApplyAsync on: " +
Thread.currentThread().getName());
 return name.toUpperCase(); // may run on different thread
 });

 System.out.println(applyAsync.get());
 }
}

```

- iii. **exceptionally()** handles exceptions and provides a fallback value.  
**handle()** can handle both success and failure cases.

```

public class ExceptionDemo {
 public static void main(String[] args) throws Exception {

 // exceptionally — catches exception and provides fallback (only on failure)
 CompletableFuture<String> withFallback = CompletableFuture
 .supplyAsync(() -> {
 if (true) throw new RuntimeException("DB connection failed");
 return "data";
 })
 .exceptionally(ex -> {

```

```

 System.out.println("exceptionally caught: " + ex.getMessage());
 return "fallback-data"; // fallback value
 });

 System.out.println("exceptionally result: " + withFallback.get());

 System.out.println("---");

 // handle — runs on BOTH success and failure (like try-catch-finally)
 CompletableFuture<String> handled = CompletableFuture
 .supplyAsync(() -> {
 if (true) throw new RuntimeException("Timeout");
 return "real-data";
 })
 .handle((result, ex) -> {
 if (ex != null) {
 System.out.println("handle caught: " + ex.getMessage());
 return "handled-fallback";
 }
 System.out.println("handle success: " + result);
 return result.toUpperCase();
 });

 System.out.println("handle result: " + handled.get());
}
}

```

- iv. **thenCompose()** is used when the next async task depends on the previous result.  
**thenCombine()** combines results from two independent async tasks.

```

public class ComposeVsCombine {

 // Simulates async DB call
 static CompletableFuture<String> fetchUser(int id) {
 return CompletableFuture.supplyAsync(() -> "User-" + id);
 }

 // Simulates async call that DEPENDS on user
 static CompletableFuture<String> fetchOrders(String user) {
 return CompletableFuture.supplyAsync(() -> "Orders-of-" + user);
 }

 // Simulates independent async call
 static CompletableFuture<String> fetchInventory() {
 return CompletableFuture.supplyAsync(() -> "Inventory-data");
 }
}

```

```

public static void main(String[] args) throws Exception {

 // thenCompose — chain dependent async tasks (flatMap equivalent)
 // fetchOrders NEEDS the result of fetchUser first
 CompletableFuture<String> composed = fetchUser(42)
 .thenCompose(user -> fetchOrders(user)); // returns CF<String>, not
 CF<CF<String>>

 System.out.println("thenCompose: " + composed.get());

 // thenCombine — run two INDEPENDENT tasks in parallel, merge results
 // fetchUser and fetchInventory run simultaneously
 CompletableFuture<String> combined = fetchUser(42)
 .thenCombine(fetchInventory(),
 (user, inventory) -> user + " " + inventory); // merge when both done

 System.out.println("thenCombine: " + combined.get());
}
}

```

v. **allOf()** waits until all given CompletableFuture tasks finish.

**anyOf()** returns when any one task finishes first.

public class ComposeVsCombine {

```

 // Simulates async DB call
 static CompletableFuture<String> fetchUser(int id) {
 return CompletableFuture.supplyAsync(() -> "User-" + id);
 }

 // Simulates async call that DEPENDS on user
 static CompletableFuture<String> fetchOrders(String user) {
 return CompletableFuture.supplyAsync(() -> "Orders-of-" + user);
 }

 // Simulates independent async call
 static CompletableFuture<String> fetchInventory() {
 return CompletableFuture.supplyAsync(() -> "Inventory-data");
 }
}

```

public static void main(String[] args) throws Exception {

```

 // thenCompose — chain dependent async tasks (flatMap equivalent)
 // fetchOrders NEEDS the result of fetchUser first
 CompletableFuture<String> composed = fetchUser(42)
 .thenCompose(user -> fetchOrders(user)); // returns CF<String>, not
 CF<CF<String>>

```

```

 System.out.println("thenCompose: " + composed.get());

 // thenCombine — run two INDEPENDENT tasks in parallel, merge results
 // fetchUser and fetchInventory run simultaneously
 CompletableFuture<String> combined = fetchUser(42)
 .thenCombine(fetchInventory(),
 (user, inventory) -> user + " + " + inventory); // merge when both done

 System.out.println("thenCombine: " + combined.get());
 }
}

```

- vi. **thenAccept()** consumes a result but does not return a new value.  
**thenRun()** runs another task after the previous task finishes, but it does not use the previous result.

```

public class AcceptVsRun {
 public static void main(String[] args) throws Exception {

 // thenAccept — USES the result, returns Void
 CompletableFuture<Void> accept = CompletableFuture
 .supplyAsync(() -> "Report-PDF")
 .thenAccept(result -> {
 // receives the result, performs side effect
 System.out.println("thenAccept: saving result → " + result);
 });
 accept.get();

 System.out.println("---");

 // thenRun — does NOT receive the result, just runs after
 CompletableFuture<Void> run = CompletableFuture
 .supplyAsync(() -> "Order-#123")
 .thenRun(() -> {
 // no access to previous result
 System.out.println("thenRun: pipeline finished, sending notification");
 });
 run.get();
 }
}

```